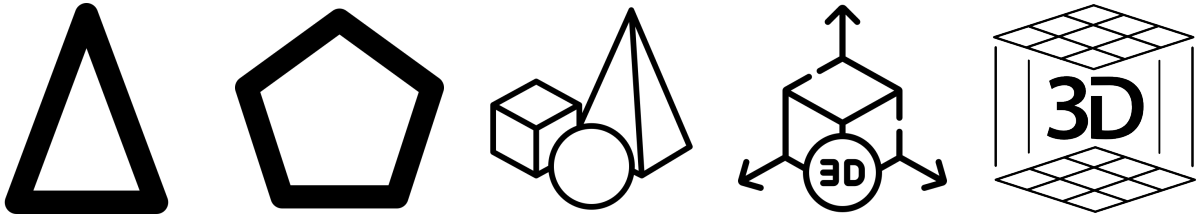


CSC 4505 (2025/26)
Graphics and Visual Computing

Lecture 5: Vertex Specification

Lighton Phiri <lighton.phiri@unza.zm>
Department of Computing and Informatics
University of Zambia
https://datalab.unza.zm/people/lightonphiri

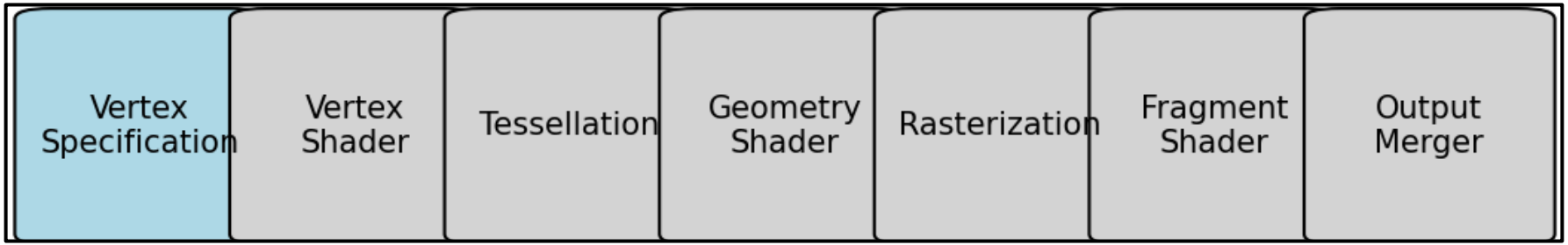
Lecture Series Outline

- Recap – The Graphics Pipeline
- Vertex Attributes – What Makes Up a Vertex?
- Understanding the Coordinate System on the Pygame Canvas
- Specifying 3D Vertices in OpenGL (Immediate Mode)
- Drawing Primitives – Points, Lines, and Triangles
- Building Complex Objects – A 3D Cube
- Complete 3D Example with Code Explanation
- Next Steps in the Pipeline
- Code Repository and File List

Lecture Series Outline

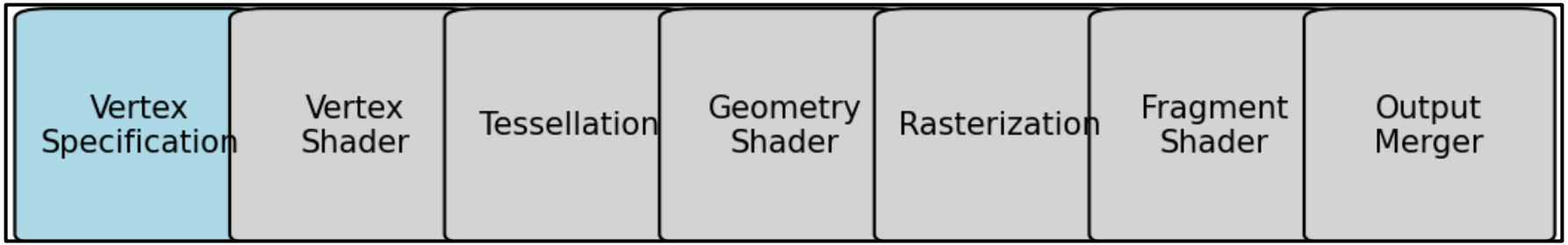
- Recap – The Graphics Pipeline
- Vertex Attributes – What Makes Up a Vertex?
- Understanding the Coordinate System on the Pygame Canvas
- Specifying 3D Vertices in OpenGL (Immediate Mode)
- Drawing Primitives – Points, Lines, and Triangles
- Building Complex Objects – A 3D Cube
- Complete 3D Example with Code Explanation
- Next Steps in the Pipeline
- Code Repository and File List

Where Are We in the Pipeline? (1/4)



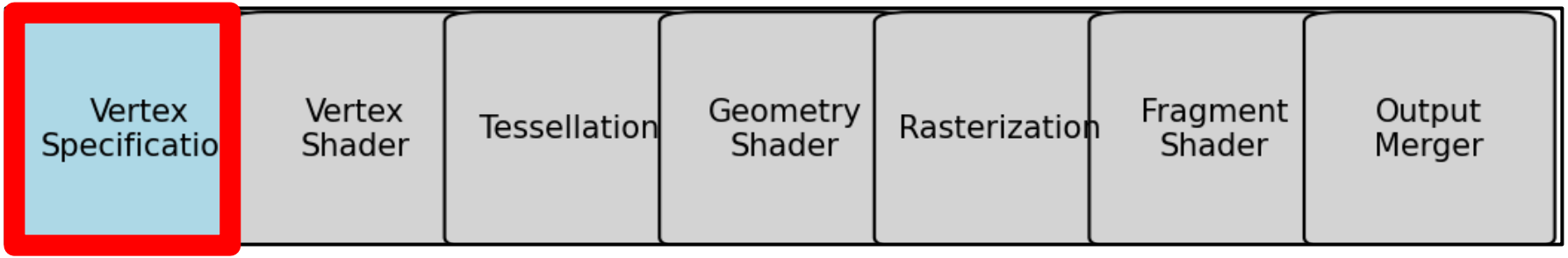
- **The graphics pipeline transforms 3D data into 2D images**
  - Vertex Specification – We provide raw vertex data (positions, colors, etc.) to the GPU
  - Vertex Shader – A small program that processes each vertex individually
  - Tessellation (optional) – Subdivides primitives
  - Geometry Shader (optional) – Can add or remove primitives

Where Are We in the Pipeline? (2/4)



- **The graphics pipeline transforms 3D data into 2D images**
  - Rasterization – Converts primitives into fragments (potential pixels)
  - Fragment Shader – Colors each fragment
  - Output Merging – Combines fragments with the framebuffer

Where Are We in the Pipeline? (3/4)



- **The graphics pipeline transforms 3D data into 2D images**
  - Vertex Specification is the very first stage: we provide raw vertex data to the GPU
- **In this lecture series, we will feed vertices directly in NDC – range [-1,1] on all axes**
- **Later lectures will add transformations to move objects through world, view, and clip spaces**

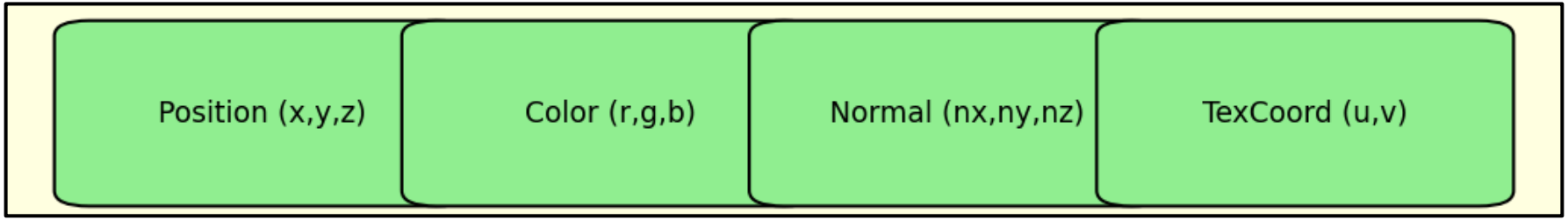
Where Are We in the Pipeline? (4/4)

- **The graphics pipeline transforms 3D data into 2D images**
  - Vertex Specification is the very first stage: we provide raw vertex data to the GPU
- **In this lecture series, we will feed vertices directly in NDC – range [-1,1] on all axes**
- **Later lectures will add transformations to move objects through world, view, and clip spaces**

Lecture Series Outline

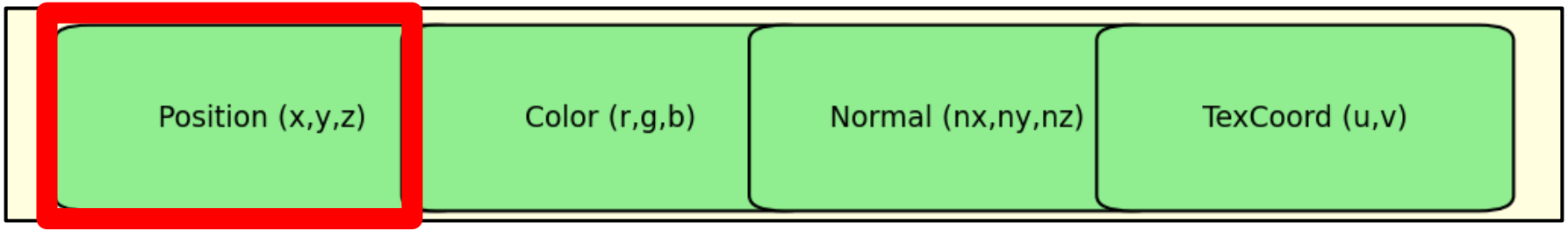
- Recap – The Graphics Pipeline
- Vertex Attributes – What Makes Up a Vertex?
- Understanding the Coordinate System on the Pygame Canvas
- Specifying 3D Vertices in OpenGL (Immediate Mode)
- Drawing Primitives – Points, Lines, and Triangles
- Building Complex Objects – A 3D Cube
- Complete 3D Example with Code Explanation
- Next Steps in the Pipeline
- Code Repository and File List

What is a Vertex?



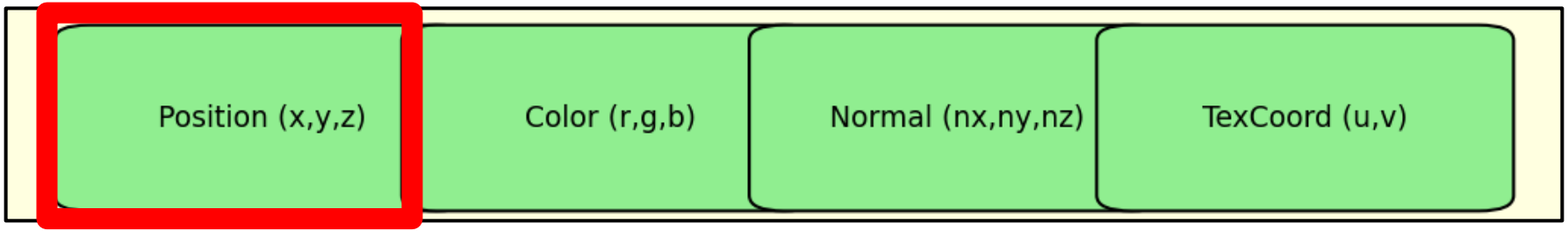
- In everyday language, a vertex is a corner point of a shape
- In computer graphics, a vertex is much richer: it is a collection of data associated with that point
- Each piece of data is called an attribute
- Attributes are inputs to the vertex shader and can be used for many purposes: positioning, coloring, lighting, texturing, animation, etc.

Vertex Attribute: Position (1/3)



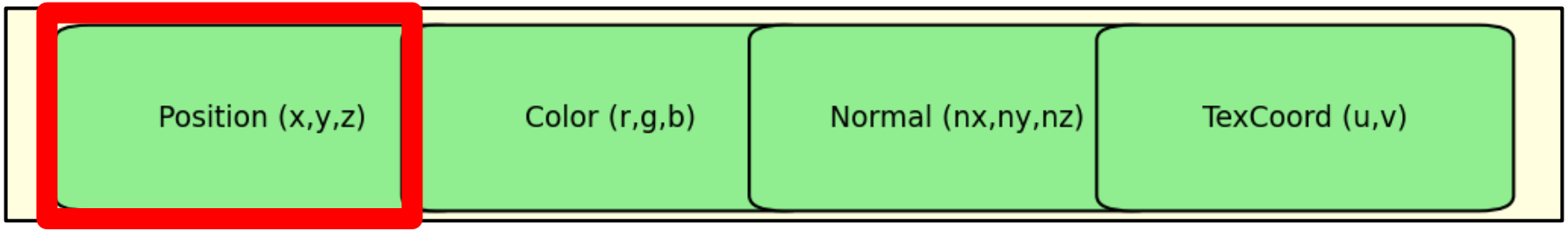
- **Position is the only mandatory attribute**
- **Usually represented as 3 components (x, y, z) or 4 components (x, y, z, w)**
- **What are x, y, z?**
  - They are coordinates in some space (initially NDC, later object/world/view/clip space)

Vertex Attribute: Position (2/3)



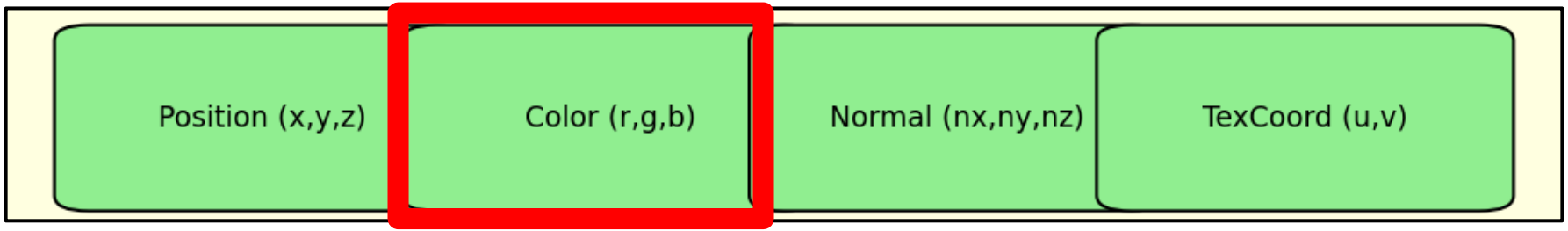
- **What are x, y, z?**
  - They are coordinates in some space (initially NDC, later object/world/view/clip space)
  - x: horizontal axis (left/right)
  - y: vertical axis (up/down)
  - z: depth axis (near/far)

Vertex Attribute: Position (3/3)



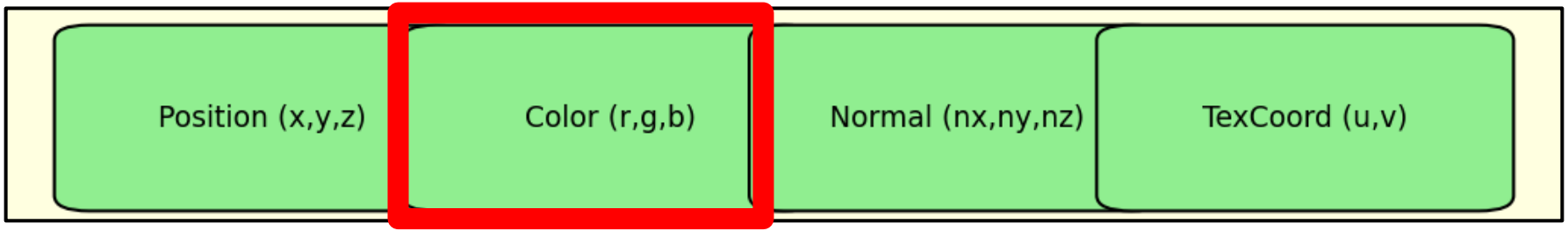
- **What is w?**
  - In homogeneous coordinates, w is a scaling factor
  - For Cartesian points, we typically set w = 1
  - Allows translation to be represented as matrix multiplication (important for transformations)
  - In clip space, w can be used for perspective division
- **Without position, GPU does not know where to place vertex**

Vertex Attribute: Color (1/)



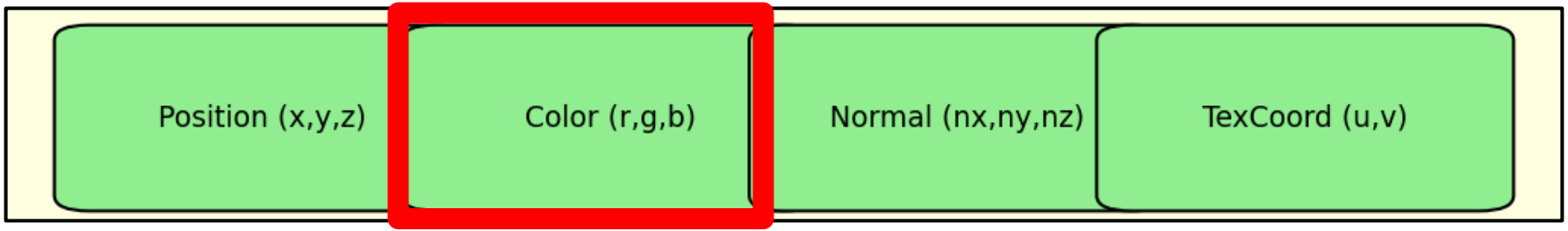
- **Color is optional but very common**
- **Usually represented as 3 components (r, g, b) or 4 components (r, g, b, a)**
- **What are r, g, b?**
  - Red, green, blue intensity values; combined to form any color
  - Typically in range [0.0, 1.0] for floating point, or [0, 255] for unsigned bytes

Vertex Attribute: Color (2/)



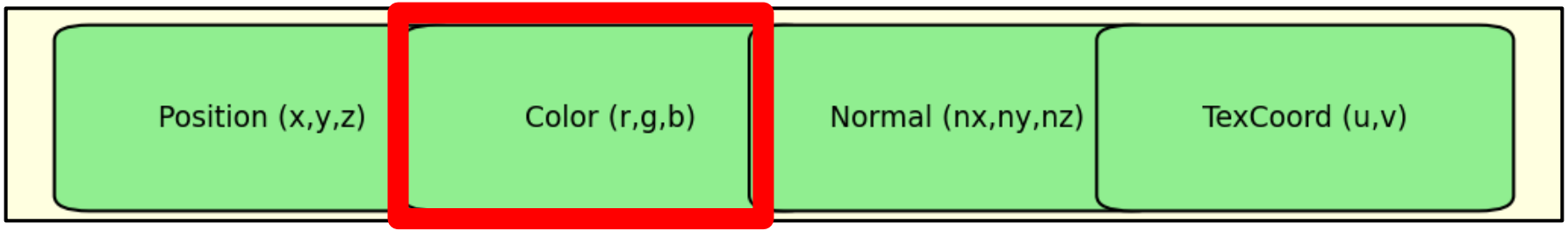
- **What is a (alpha)?**
  - Alpha channel: transparency
  - 0.0 = fully transparent, 1.0 = fully opaque
  - Used for blending, translucency, etc

Vertex Attribute: Color (3/)



- **Why is "Color" important?**
  - Provides per-vertex color that can be interpolated across the triangle
  - Enables smooth color gradients (e.g., red to green)
  - Base for many shading effects

Vertex Attribute: Color (4/)



- **Each component is a floating-point value between 0.0 and 1.0.**
  - 0.0 means no contribution of that colour
  - 1.0 means maximum intensity

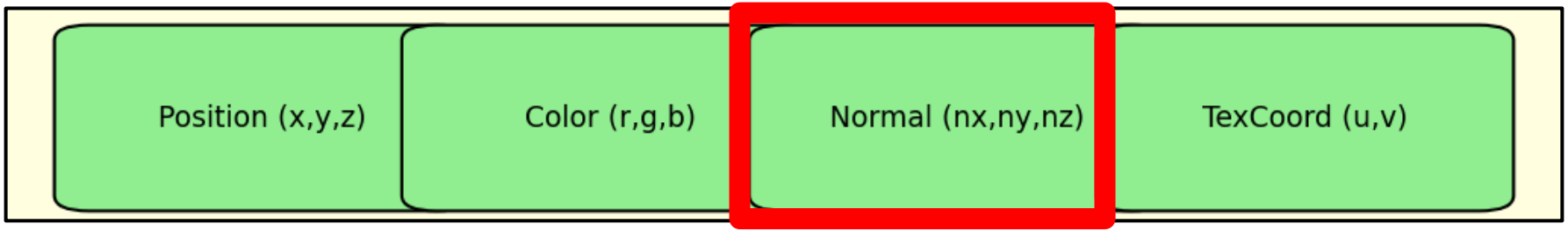
Vertex Attribute: Color (5/)

- **Common colour examples**
  - Red: (1.0, 0.0, 0.0)
  - Green: (0.0, 1.0, 0.0)
  - Blue: (0.0, 0.0, 1.0)
  - Yellow: (1.0, 1.0, 0.0) (red + green)
  - Cyan: (0.0, 1.0, 1.0) (green + blue)
  - Magenta:(1.0, 0.0, 1.0) (red + blue)
  - White: (1.0, 1.0, 1.0)
  - Black: (0.0, 0.0, 0.0)

Vertex Attribute: Color (6/)

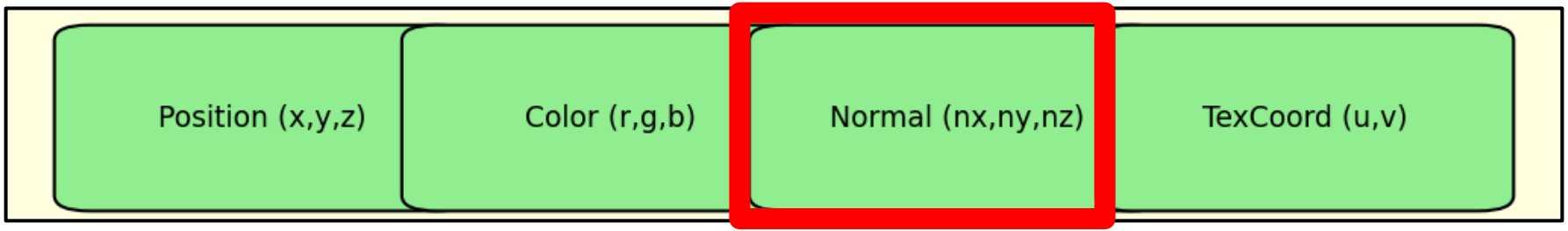
- In OpenGL immediate mode, we set the current colour with glColor3f(r, g, b)
- All vertices drawn after this call will use that colour until we change it

Vertex Attribute: Normal (1/3)



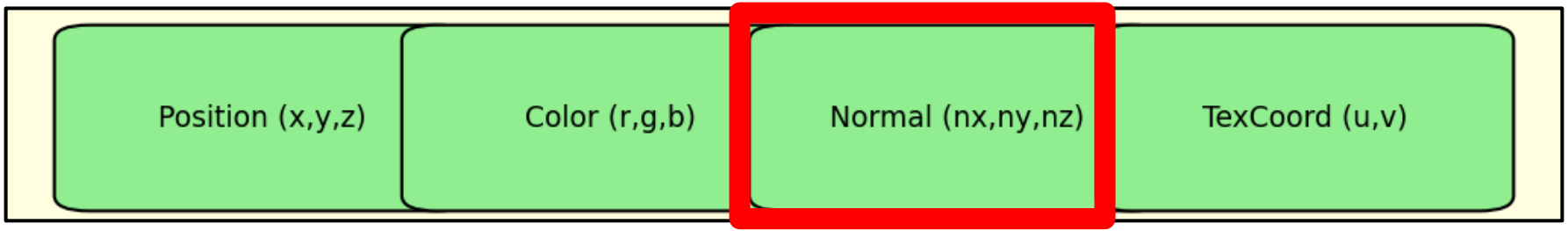
- **Normal is optional but essential for lighting.**
- **Represented as 3 components (nx, ny, nz).**
- **What is a normal vector?**
  - A unit vector (length = 1) perpendicular to the surface at the vertex
  - Points outward from the surface

Vertex Attribute: Normal (2/3)



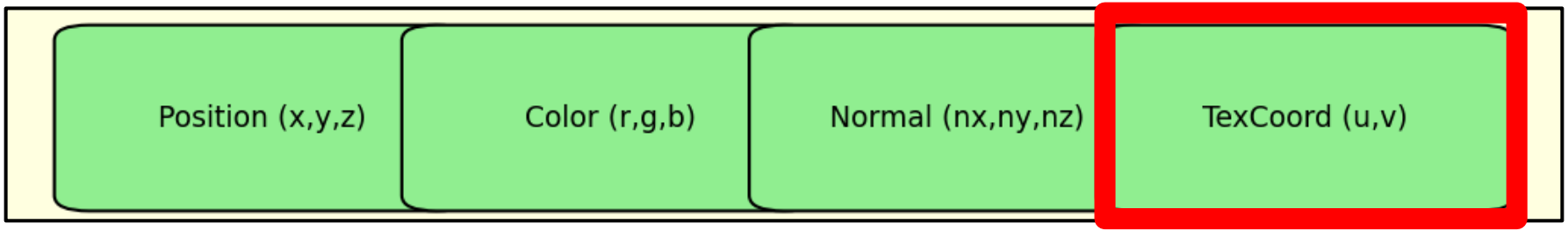
- **Why is the "Normal" important?**
  - Lighting calculations depend on the angle between the light direction and the normal
  - Example: If light hits a surface straight on (angle 0°), it's brightest; if it grazes the surface (90°), it's dark
  - Without normals, we cannot do realistic shading (flat shading would look unnatural)

Vertex Attribute: Normal (3/3)



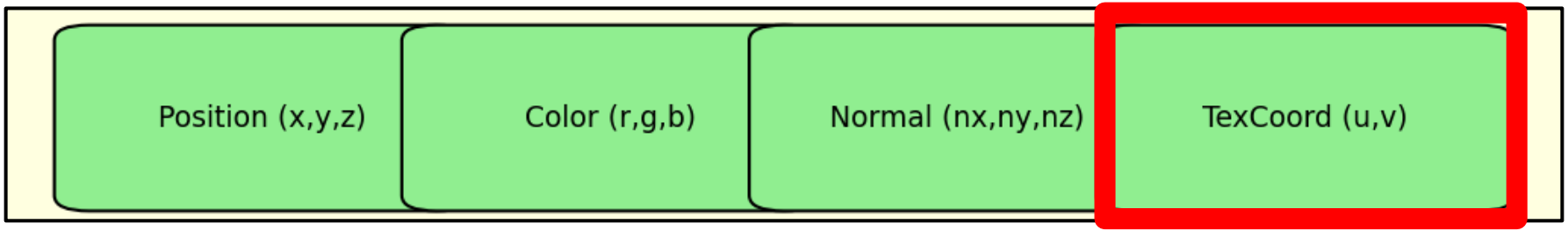
- **Why is the "Normal" important?**
  - Normals are interpolated across the triangle for smooth lighting (Phong shading)

Vertex Attribute: Texture Coordinates (1/3)



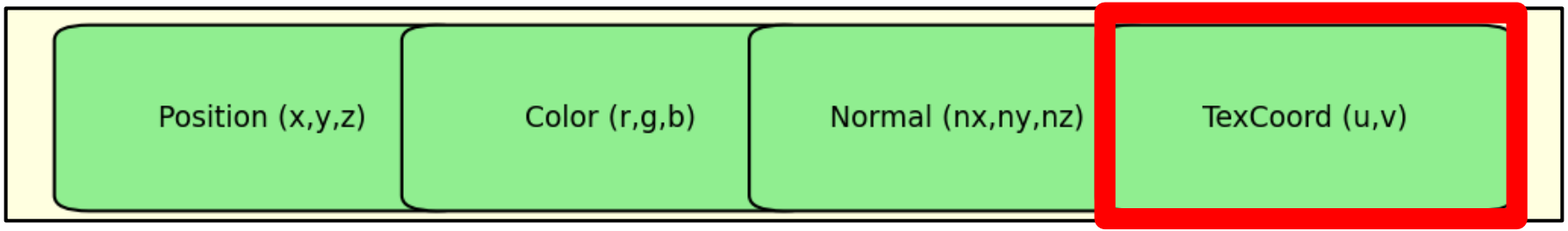
- Texture coordinates (also called UVs) are optional but essential for texturing
- Usually represented as 2 components (u, v); sometimes 3 for 3D textures

Vertex Attribute: Texture Coordinates (2/3)



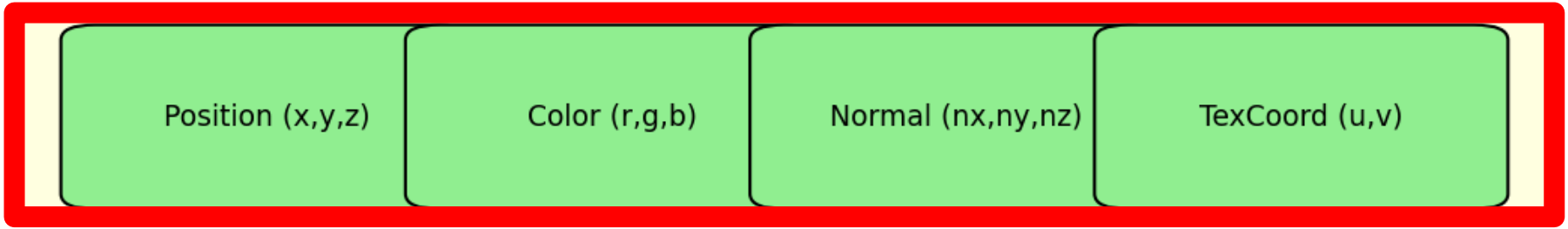
- **What are u and v?**
  - They are coordinates in a 2D texture image
  - u = horizontal coordinate (0 = left edge, 1 = right edge)
  - v = vertical coordinate (0 = bottom edge, 1 = top edge)
    - Note: In some graphics systems, v=0 is top, but OpenGL uses bottom as 0 by default; it can be flipped

Vertex Attribute: Texture Coordinates (3/3)



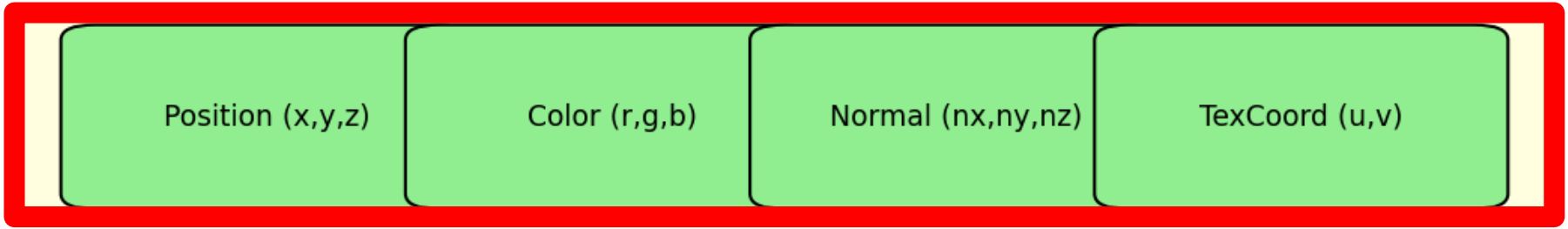
- **Why are "Texture Coordinates" important?**
  - They map a 2D image (texture) onto a 3D surface
  - Each vertex gets a (u,v) pair; during rasterization, these are interpolated so every fragment knows which texel (texture pixel) to sample
  - Enables detailed surface appearance without adding more geometry (e.g., brick wall, grass, skin)

Vertex Attribute: Custom Attributes



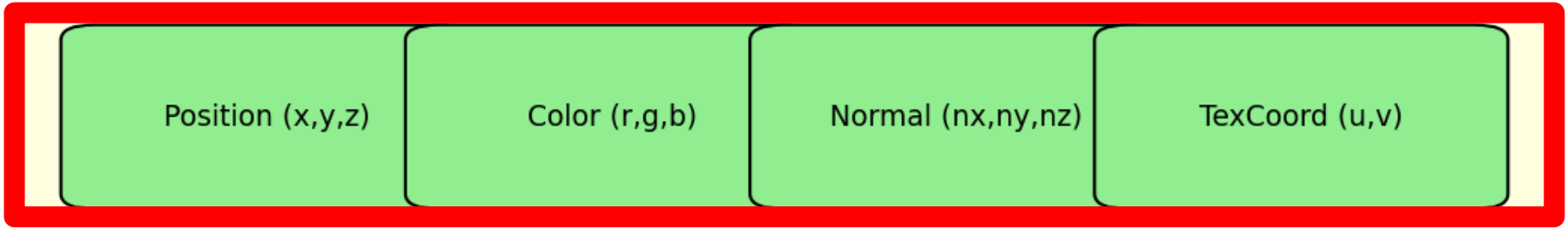
- **OpenGL allows you to define any number of attributes (up to a hardware limit, typically ≥16)**
  - Custom attributes give enormous flexibility
- **Some Examples**
  - Bone weights and bone indices for skeletal animation (skinning)
  - Tangents and bitangents for normal mapping
  - Vertex IDs for picking or special effects

Vertex Attribute: Attribute Locations (1/2)



- **In the vertex shader, each input attribute is identified by a location index**
  - We assign these indices when setting up attribute pointers
  - The shader declares: layout(location = 0) in vec3 aPos;
  - This connects the data in the VBO to the shader variable

Vertex Attribute: Attribute Locations (2/2)



- **Example mapping**
  - location 0 = position
  - location 1 = color
  - location 2 = normal
  - location 3 = texcoord

Lecture Series Outline

- Recap – The Graphics Pipeline
- Vertex Attributes – What Makes Up a Vertex?
- Understanding the Coordinate System on the Pygame Canvas
- Specifying 3D Vertices in OpenGL (Immediate Mode)
- Drawing Primitives – Points, Lines, and Triangles
- Building Complex Objects – A 3D Cube
- Complete 3D Example with Code Explanation
- Next Steps in the Pipeline
- Code Repository and File List

Understanding the Pygame Canvas (1/2)

- **When we draw using NDC coordinates, the GPU automatically maps the NDC cube to the viewport (the Pygame window)**
- **This mapping is called the viewport transformation**
- **For a window of size 800×600**
  - NDC x = -1 maps to screen x = 0
  - NDC x = +1 maps to screen x = 800
  - Similarly for y (with possible inversion depending on coordinate system)

Understanding the Pygame Canvas (2/2)

- Important: The coordinates we specify are in model space (for now, we place objects directly in NDC). The GPU transforms them through the pipeline; by the time they appear on the canvas, they have been rasterised and are in screen space
- In immediate mode, we are effectively providing vertices in NDC, and OpenGL handles the rest

Lecture Series Outline

- Recap – The Graphics Pipeline
- Vertex Attributes – What Makes Up a Vertex?
- Understanding the Coordinate System on the Pygame Canvas
- Specifying 3D Vertices in OpenGL (Immediate Mode)
- Drawing Primitives – Points, Lines, and Triangles
- Building Complex Objects – A 3D Cube
- Complete 3D Example with Code Explanation
- Next Steps in the Pipeline
- Code Repository and File List

Drawing Primitives: Points, Lines, Triangles

- Now that we understand what a vertex is, we need to understand what we can draw with them.
- In OpenGL, vertices are assembled into primitives: points, lines, and triangles.
- The type of primitive is specified when we issue a drawing command.

Drawing Primitives: Immediate Mode—The Simplest Way to Draw (1/4)

- Before diving into modern OpenGL, we'll start with immediate mode (legacy OpenGL)
- Immediate mode uses glBegin() and glEnd() to specify vertices directly
- Why start here? It makes the concept of vertex specification concrete without the complexity of buffers and shaders.

Drawing Primitives: Immediate Mode—The Simplest Way to Draw (2/4)

- glBegin(primitive_type) marks the start of a list of vertices that will form one or more primitives
- The primitive_type argument tells OpenGL how to interpret the vertices that follow
- After glBegin, we call glVertex* functions to specify vertices

Drawing Primitives: Immediate Mode—The Simplest Way to Draw (3/4)

- Common primitive types

Module	Contents
GL_POINTS	Each vertex is a point
GL_LINES	Each pair of vertices forms a line segment
GL_LINE_STRIP	Vertices form a connected polyline
GL_LINE_LOOP	Similar to strip but connects last vertex back to first
GL_TRIANGLES	Each triple of vertices forms a triangle
GL_TRIANGLE_STRIP	Vertices form a strip of triangles
GL_TRIANGLE_FAN	First vertex is hub, subsequent pairs form triangles

Drawing Primitives: Immediate Mode—The Simplest Way to Draw (4/4)

- glEnd() marks the end of a vertex list
- It signals that all vertices for the current primitive have been specified
- Every glBegin must be matched with a glEnd.
- Between them, only certain OpenGL functions are allowed: glVertex, glColor, glNormal, glTexCoord, etc
- Calling other functions (like matrix operations) between glBegin and glEnd is illegal and may cause errors

Drawing Primitives: The glVertex Family of Functions (1/2)

- **OpenGL provides many variants of glVertex to handle different data types and dimensions**
- **Naming convention: glVertex{2|3}{s|i|f|d}{v}**
  - 2 or 3 – number of dimensions (x,y) or (x,y,z)
  - s – short integer (16-bit)
  - i – integer (32-bit)
  - f – float (32-bit)
  - d – double (64-bit)
  - v – vector (pointer to an array) – the "v" suffix

Drawing Primitives: The glVertex Family of Functions (2/2)

- **OpenGL provides many variants of glVertex to handle different data types and dimensions**
- **Naming convention: glVertex{2|3}{s|i|f|d}{v}**
  - 2 or 3 – number of dimensions (x,y) or (x,y,z)
  - s – short integer (16-bit)
  - i – integer (32-bit)
  - f – float (32-bit)
  - d – double (64-bit)
  - v – vector (pointer to an array) – the "v" suffix

Drawing Primitives: Commonly Used glVertex Variants (with examples) (1/3)

- **glVertex2f(x, y)**
  - 2D vertex using floats
  - glVertex2f(0.5, -0.5)
- **glVertex3f(x, y, z)**
  - 3D vertex using floats
  - glVertex3f(0.0, 0.0, 1.0)
- **glVertex2i(x, y)**
  - 2D vertex using integers
  - glVertex2i(100, 200)
  - Note NDC expects [-1,1] |

Drawing Primitives: Commonly Used glVertex Variants (with examples) (2/3)

- **glVertex3d(x, y, z)**
  - 3D vertex using doubles
  - glVertex3d(0.5, -0.2, 0.0)
- **glVertex2fv([x, y])**
  - 2D vertex from a list/tuple of floats
  - glVertex2fv([0.5, -0.5])
- **glVertex3fv([x, y, z])**
  - 3D vertex from a list/tuple
  - glVertex3fv([0.0, 0.0, 1.0])

Drawing Primitives: Commonly Used glVertex Variants (with examples) (3/3)

- **glVertex2iv([x, y])**
  - 2D vertex from integer list
  - glVertex2iv([100, 200])
- **glVertex3sv([x, y, z])**
  - 3D vertex from short integer list (rarely used)
- **The vector versions (glVertex* v) are useful when vertex coordinates are stored in arrays**

Drawing Primitives: Why Multiple Data Types?

- **Precision**
  - Use float (or double) for world coordinates, int or short for screen-space coordinates in some contexts
- **Memory**
  - Integers and shorts consume less memory than floats, but may require conversion
- **In immediate mode, OpenGL converts all types to floats internally anyway**
- **In modern OpenGL with VBOs, you can use any type, but you must tell OpenGL the type via glVertexAttribPointer**

Drawing Primitives: Immediate Mode Syntax (1/2)

- **glBegin -> glEnd**

```
glBegin(primitive_type)
    # Set color for subsequent vertices
    glColor3f(r, g, b)
    # Specify vertices
    glVertex2f(x, y)    # 2D vertex
    glVertex3f(x, y, z) # 3D vertex
    # ... more vertices ...
glEnd()
```

Drawing Primitives: Immediate Mode Syntax (2/2)

- **primitive_type**
  - Determines how vertices are assembled
- **Colors set with glColor apply to all vertices until changed**

Drawing Primitives: Examples of Using Different glVertex Variants (1/3)

- 2D float

```
# Using 2D float coordinates
glBegin(GL_POINTS)
glVertex2f(-0.5, -0.5)
glVertex2f( 0.5,  0.5)
glEnd()
```

Drawing Primitives: Examples of Using Different glVertex Variants (2/3)

- 3D float coordinates

```
# Using 3D float coordinates (z=0)
glBegin(GL_LINES)
glVertex3f(-0.8, -0.8, 0.0)
glVertex3f( 0.8,  0.8, 0.0)
glEnd()
```

- Using vector versions

Drawing Primitives: Examples of Using Different glVertex Variants (3/3)

- Using vector versions

```
# Using vector versions
var_points = [(-0.5, 0.5), (0.5, 0.5), (0.5, -0.5), (-0.5, -0.5)]
glBegin(GL_LINE_LOOP)
for var_p in var_points:
    glVertex2fv(var_p)   # pass tuple directly
glEnd()
```

Drawing Primitives: Setting Colors with glColor (1/2)

- In immediate mode, we can set the current color with glColor3f(r, g, b) or glColor4f(r, g, b, a)
- The color applies to all vertices specified after it, until changed

Drawing Primitives: Setting Colors with glColor (2/2)

- Example 1. Produces a triangle with red, green, and blue vertices – colors are interpolated across the triangle

```
glBegin(GL_TRIANGLES)
glColor3f(1.0, 0.0, 0.0)   # red
glVertex2f(-0.5, -0.5)
glColor3f(0.0, 1.0, 0.0)   # green
glVertex2f( 0.5, -0.5)
glColor3f(0.0, 0.0, 1.0)   # blue
glVertex2f( 0.0,  0.5)
glEnd()
```

Drawing Primitives: Our Base Pygame/OpenGL Template (1/5)

```
import pygame
from pygame.locals import *
from OpenGL.GL import *

def fxn_draw_primitives():
    # Drawing code will go here
    pass

def main():
    var_running = True
    var_display = (800, 600)
```

Drawing Primitives: Our Base Pygame/OpenGL Template (2/5)

```
pygame.init()
pygame.display.set_mode(var_display, DOUBLEBUF | OPENGL |
RESIZABLE)
pygame.display.set_caption("Immediate Mode Demo")
var_clock = pygame.time.Clock()

print("OpenGL version:", glGetString(GL_VERSION).decode())
print("Vendor:", glGetString(GL_VENDOR).decode())
print("Renderer:", glGetString(GL_RENDERER).decode())
```

Drawing Primitives: Our Base Pygame/OpenGL Template (3/5)

```
    while var_running:
        for var_event in pygame.event.get():
            if var_event.type == QUIT:
                var_running = False
            elif var_event.type == KEYDOWN and var_event.key ==
K_ESCAPE:
                var_running = False
            elif var_event.type == VIDEORESIZE:
                var_display = var_event.size
                pygame.display.set_mode(var_display, DOUBLEBUF |
OPENGL | RESIZABLE)
```

Drawing Primitives: Our Base Pygame/OpenGL Template (4/5)

```
        glViewport(0, 0, var_display[0], var_display[1])

    glClearColor(0.2, 0.3, 0.4, 1.0)
    glClear(GL_COLOR_BUFFER_BIT)

    fxn_draw_primitives()

    pygame.display.flip()
    var_clock.tick(60)
```

Drawing Primitives: Our Base Pygame/OpenGL Template (5/5)

```
    pygame.quit()

if __name__ == "__main__":
    main()
```

Lecture Series Outline

- Recap – The Graphics Pipeline
- Vertex Attributes – What Makes Up a Vertex?
- Understanding the Coordinate System on the Pygame Canvas
- Specifying 3D Vertices in OpenGL (Immediate Mode)
- Drawing Primitives – Points, Lines, and Triangles
- Building Complex Objects – A 3D Cube
- Complete 3D Example with Code Explanation
- Next Steps in the Pipeline
- Code Repository and File List

Drawing Primitives: Drawing Points (GL_POINTS) (1/4)

- Primitive type: GL_POINTS
- Each vertex becomes a single point
- Point size controlled by glPointSize(size)
- Full code available in csc4505_lecture5_points.py (GitHub)

Drawing Primitives: Drawing Points (GL_POINTS) (2/4)

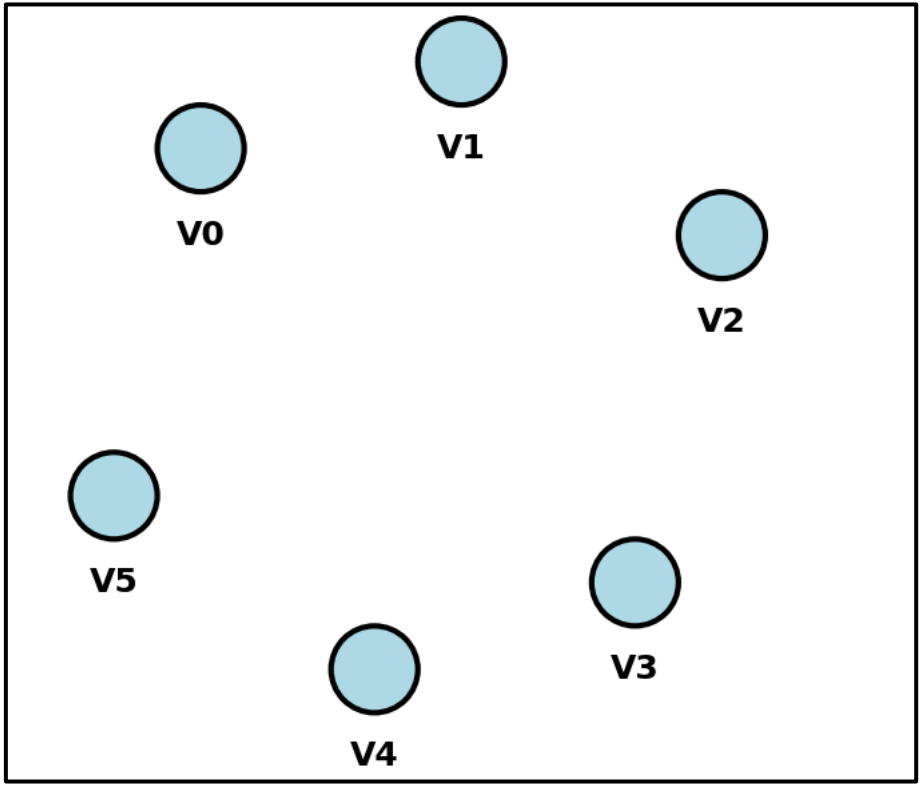
- **Each vertex becomes a single point**
  - Control size with glPointSize()

```
def fxn_draw_primitives():
    # Draw four white points at the corners
    glPointSize(5.0)
    glBegin(GL_POINTS)
    glColor3f(1.0, 1.0, 1.0)  # white
    glVertex2f(-0.8,  0.8)    # top-left
    glVertex2f( 0.8,  0.8)    # top-right
    glVertex2f( 0.8, -0.8)    # bottom-right
    glVertex2f(-0.8, -0.8)    # bottom-left
    glEnd()
```

Drawing Primitives: Drawing Points (GL_POINTS) (3/4)

- **Primitive type: GL_POINTS**
  - glPointSize(5.0): Makes points larger than the default 1 pixel
  - glBegin(GL_POINTS): Tells OpenGL to treat each vertex as an independent point
  - glColor3f(1.0, 1.0, 1.0): Sets the current colour to white; all vertices until the next colour change will be white
  - glVertex3f(...): Each call specifies a vertex at the given NDC coordinates
  - glEnd(): Finishes the point list; the GPU renders the four points
- **Four white points appear at the corners of the window (in screen space after viewport mapping)**

Drawing Primitives: Drawing Points (GL_POINTS) (4/4)



Drawing Primitives: Drawing Lines (GL_LINES) (1/4)

- Primitive type: GL_LINES
- Each pair of vertices forms an independent line segment
- With 4 vertices, we get two lines: (v0,v1) and (v2,v3)
- Line width controlled by glLineWidth(width)
- Full code available in csc4505_lecture5_lines.py (GitHub)

Drawing Primitives: Drawing Lines (GL_LINES) (2/4)

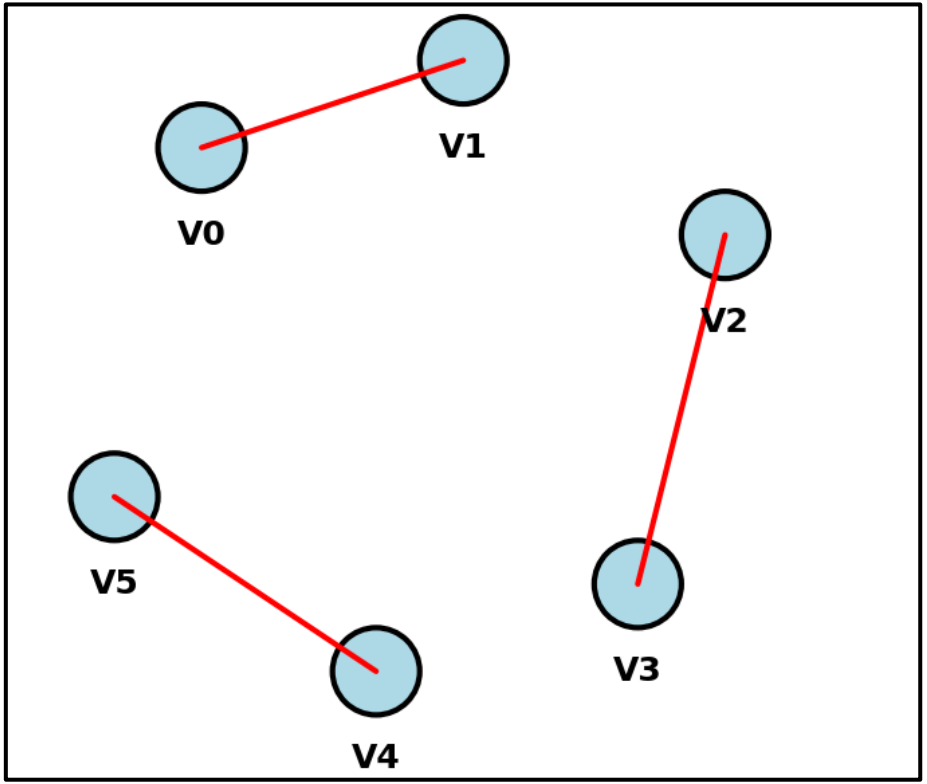
- Each pair of vertices forms an independent line segment

```
def fxn_draw_primitives():
    # Draw two diagonal green lines
    glLineWidth(2.0)
    glBegin(GL_LINES)
    glColor3f(0.0, 1.0, 0.0)  # green
    glVertex2f(-0.5,  0.5)    # start line 1
    glVertex2f( 0.5, -0.5)    # end line 1
    glVertex2f(-0.5, -0.5)    # start line 2
    glVertex2f( 0.5,  0.5)    # end line 2
    glEnd()
```

Drawing Primitives: Drawing Lines (GL_LINES) (3/4)

- **Explanation**
  - glLineWidth(2.0): Makes lines thicker for visibility
  - glBegin(GL_LINES): Each two vertices define a separate line
  - glColor3f(0.0,1.0,0.0): Sets colour to green for all following vertices
  - The first two vertices define one diagonal line from top-left to bottom-right
  - The next two vertices define another diagonal line from bottom-left to top-right
  - glEnd(): Completes the lines; GPU draws two green lines crossing to form an X
  - ***If an odd number of vertices is given, the last one is ignored***

Drawing Primitives: Drawing Lines (GL_LINES) (4/4)



Drawing Primitives: Drawing Line Strips (GL_LINE_STRIP) (1/5)

- Primitive type: GL_LINE_STRIP
- Vertices are connected in sequence: v0–v1, v1–v2, v2–v3, … forming a continuous polyline
- Last vertex does not connect back to first
- Full code available in csc4505_lecture5_line_strip.py (GitHub)

Drawing Primitives: Drawing Line Strips (GL_LINE_STRIP) (2/5)

- Vertices connected in sequence: v0-v1, v1-v2, v2-v3, ...

```
def fxn_draw_primitives(): # Draw a yellow zigzag line
    glBegin(GL_LINE_STRIP)
    glColor3f(1.0, 1.0, 0.0)  # yellow
    glVertex2f(-0.9,  0.2)
    glVertex2f(-0.6,  0.5)
    glVertex2f(-0.3,  0.2)
    glVertex2f( 0.0,  0.5)
    glVertex2f( 0.3,  0.2)
```

Drawing Primitives: Drawing Line Strips (GL_LINE_STRIP) (3/5)

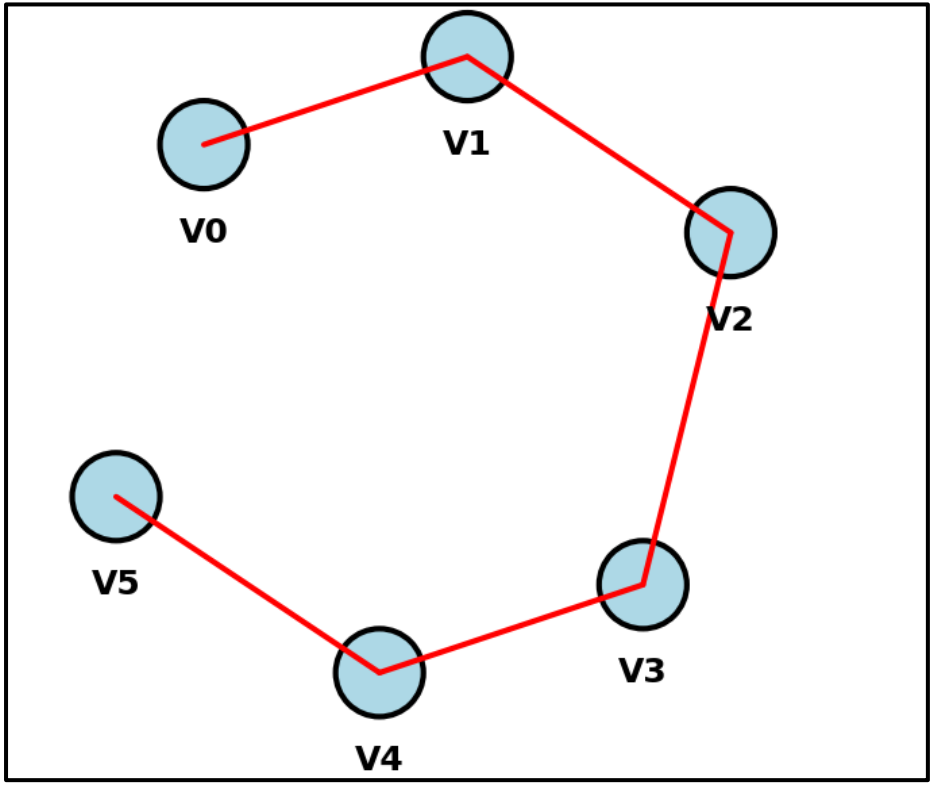
- Vertices connected in sequence: v0-v1, v1-v2, v2-v3, ...

```
def fxn_draw_primitives(): # Draw a yellow zigzag line
    # [...]
    # [...]
    glVertex2f( 0.9,  0.2)
    glEnd()
```

Drawing Primitives: Drawing Line Strips (GL_LINE_STRIP) (4/5)

- **Explanation**
  - glBegin(GL_LINE_STRIP): Vertices will be connected in order
  - glColor3f(1,1,0): Sets colour to yellow for the entire strip
  - Each glVertex3f adds a vertex. OpenGL connects v0–v1, v1–v2, …, v5–v6
  - The result is a continuous zigzag line
- **Example use cases: Drawing polylines, graphs, or any connected sequence**

Drawing Primitives: Drawing Line Strips (GL_LINE_STRIP) (5/5)



Drawing Primitives: Drawing Line Loops (GL_LINE_LOOP) (1/4)

- Primitive type: GL_LINE_LOOP
- Like a line strip, but also connects the last vertex back to the first, forming a closed loop
- Full code available in csc4505_lecture5_line_loop.py (GitHub)

Drawing Primitives: Drawing Line Loops (GL_LINE_LOOP) (2/4)

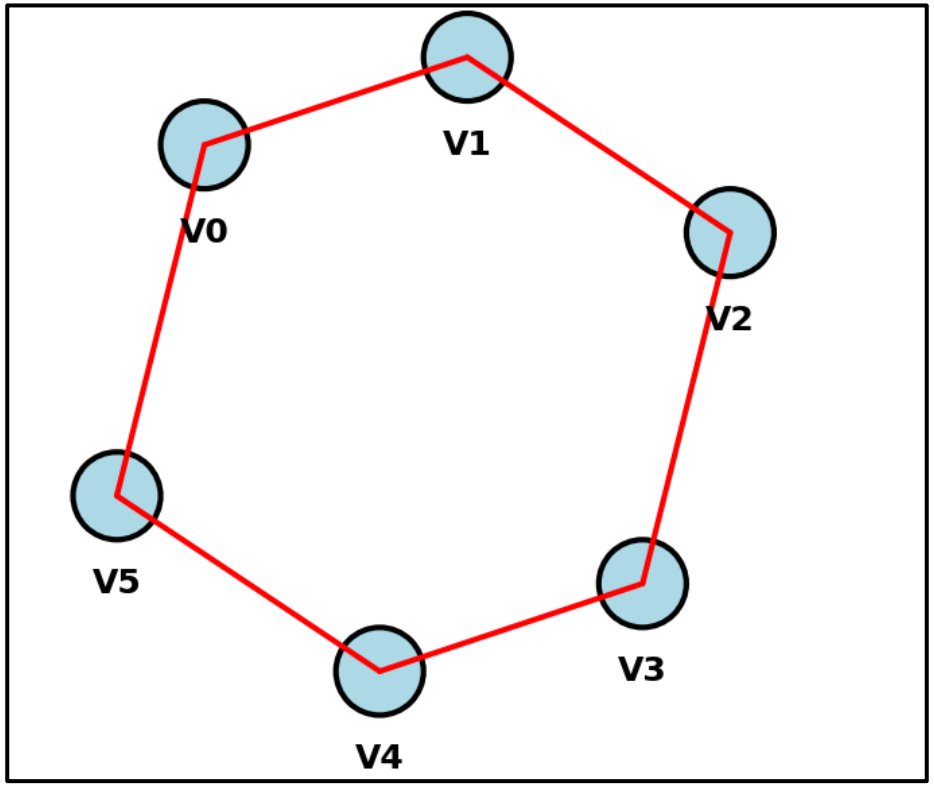
- Like line strip, but also connects last vertex back to first

```
def fxn_draw_primitives():
    # Draw a cyan triangle outline
    glBegin(GL_LINE_LOOP)
    glColor3f(0.0, 1.0, 1.0)  # cyan
    glVertex2f(-0.7, -0.2)
    glVertex2f( 0.7, -0.2)
    glVertex2f( 0.0, -0.7)
    glEnd()
```

Drawing Primitives: Drawing Line Loops (GL_LINE_LOOP) (3/4)

- **Explanation**
  - glBegin(GL_LINE_LOOP): Vertices are connected in order, and then v2 is connected back to v0
  - The three vertices define a triangle outline
- **Example use cases: Drawing wireframe polygons without filling them**

Drawing Primitives: Drawing Line Loops (GL_LINE_LOOP) (4/4)



Drawing Primitives: Drawing Triangles (GL_TRIANGLES) (1/4)

- Primitive type: GL_TRIANGLES
- Each triple of vertices forms an independent, filled triangle
- This is the most important primitive for 3D graphics
- Full code available in csc4505_lecture5_triangles.py (GitHub)

Drawing Primitives: Drawing Triangles (GL_TRIANGLES) (2/4)

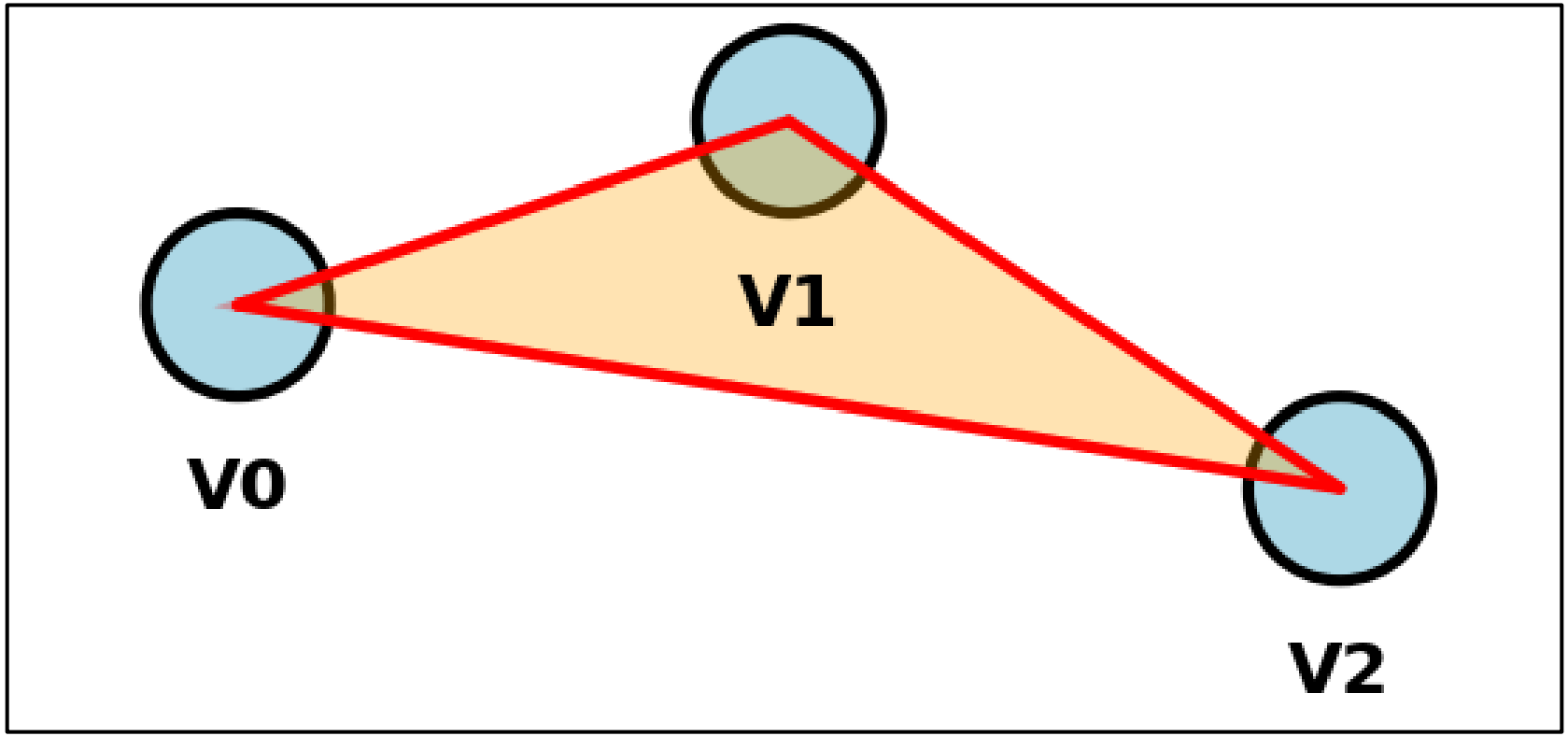
- Each triple of vertices forms a filled triangle

```
def fxn_draw_primitives():
    # Draw a red filled triangle
    glBegin(GL_TRIANGLES)
    glColor3f(1.0, 0.0, 0.0)  # red
    glVertex2f(-0.8, -0.8)
    glVertex2f(-0.2, -0.8)
    glVertex2f(-0.5, -0.3)
    glEnd()
```

Drawing Primitives: Drawing Triangles (GL_TRIANGLES) (3/4)

- **Explanation**
  - glBegin(GL_TRIANGLES): Each three vertices make one triangle
  - We set a different colour before each vertex
  - OpenGL interpolates the colours across the triangle, creating a smooth gradient from red to green to blue
- **Why triangles? Any 3D surface can be approximated by triangles, and the GPU is highly optimised for them**

Drawing Primitives: Drawing Triangles (GL_TRIANGLES) (4/4)



Drawing Primitives: Drawing Triangle Fans (GL_TRIANGLE_FAN) (1/4)

- Primitive type: GL_TRIANGLE_FAN
- The first vertex is the hub. Each subsequent pair forms a triangle with the hub: (v0,v1,v2), (v0,v2,v3), ...
- Efficient for shapes that share a common point (e.g., a filled circle)
- Full code available in csc4505_lecture5_triangle_fan.py (GitHub)

Drawing Primitives: Drawing Triangle Fans (GL_TRIANGLE_FAN) (2/4)

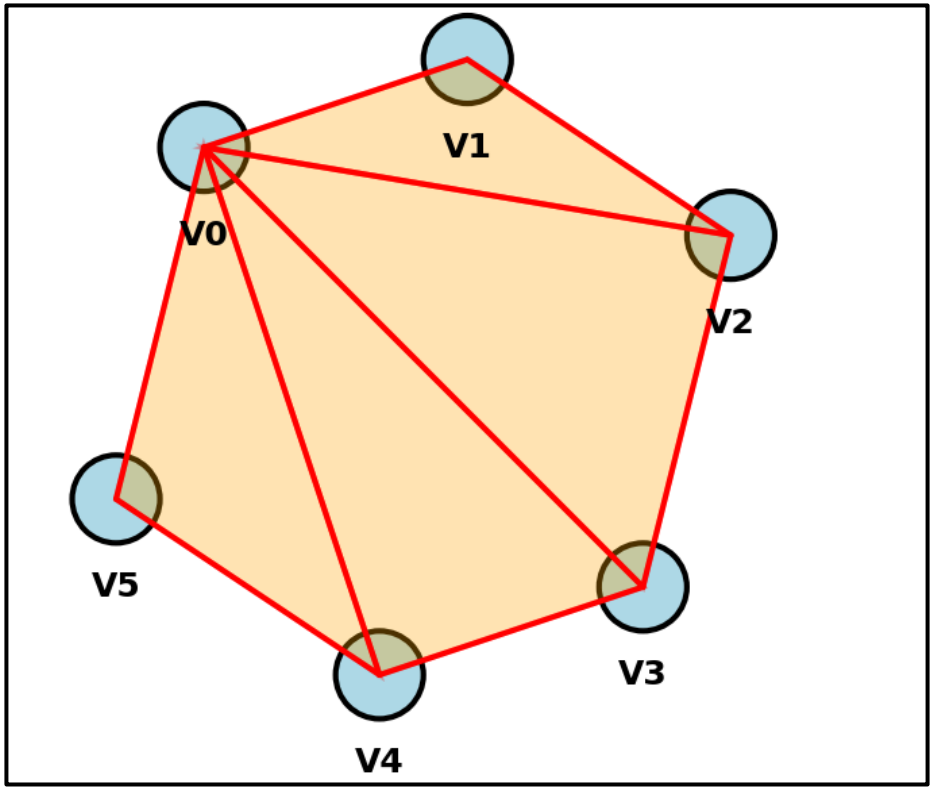
- First vertex is hub; each subsequent pair forms triangle with hub

```
def fxn_draw_primitives():
    # Draw a blue quadrilateral as a triangle fan
    glBegin(GL_TRIANGLE_FAN)
    glColor3f(0.0, 0.0, 1.0)  # blue
    glVertex2f(0.5, -0.5)     # hub
    glVertex2f(0.8, -0.5)     # v1
    glVertex2f(0.8, -0.2)     # v2
    glVertex2f(0.5, -0.2)     # v3
    glEnd()
```

Drawing Primitives: Drawing Triangle Fans (GL_TRIANGLE_FAN) (3/4)

- **Explanation**
  - glBegin(GL_TRIANGLE_FAN): The first vertex (0.5,-0.5) is the hub
  - Triangles formed: (hub,v1,v2) and (hub,v2,v3). This fills a quadrilateral
- **Example use case: Drawing cones, disks, or any shape with a central point**

Drawing Primitives: Drawing Triangle Fans (GL_TRIANGLE_FAN) (4/4)



Drawing Primitives: Drawing Triangle Strips (GL_TRIANGLE_STRIP) (1/4)

- Primitive type: GL_TRIANGLE_STRIP
- Vertices form a strip of triangles: (v0,v1,v2), (v1,v2,v3), (v2,v3,v4), ...
- Very efficient for long, connected surfaces (e.g., terrain strips)
- Full code available in csc4505_lecture5_triangle_strip.py (GitHub)

Drawing Primitives: Drawing Triangle Strips (GL_TRIANGLE_STRIP) (2/4)

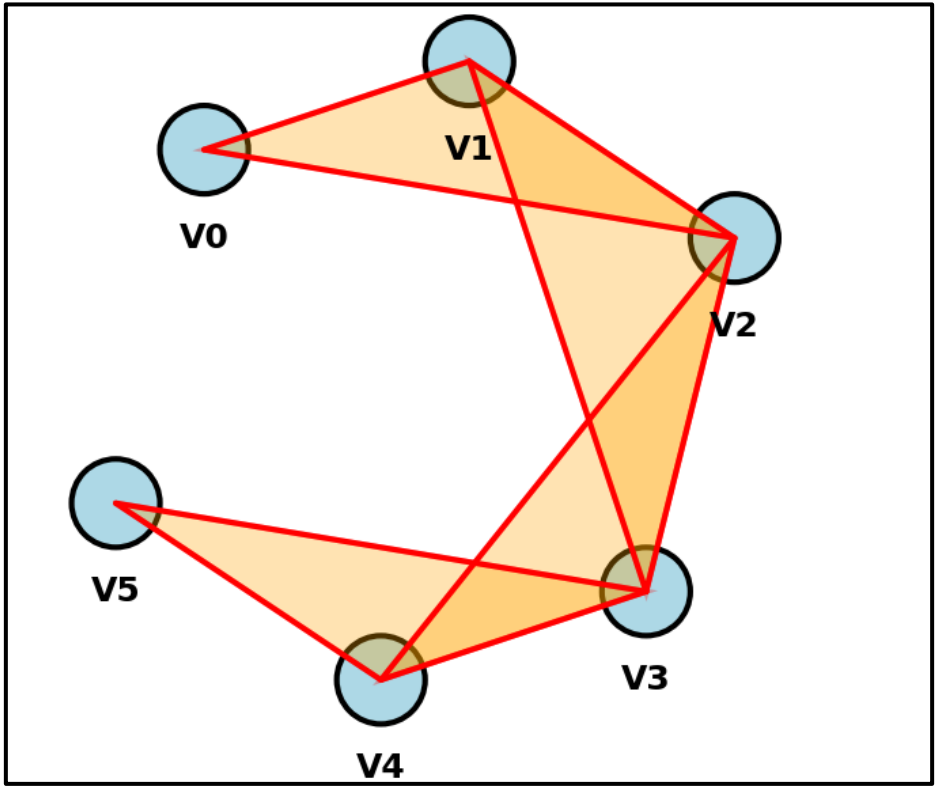
- Forms triangles: (v0,v1,v2), (v1,v2,v3), (v2,v3,v4), ...

```
def fxn_draw_primitives():
    # Draw a magenta shape with triangle strip
    glBegin(GL_TRIANGLE_STRIP)
    glColor3f(1.0, 0.0, 1.0)  # magenta
    glVertex2f(0.2, 0.2)      # v0
    glVertex2f(0.5, 0.5)      # v1
    glVertex2f(0.5, 0.2)      # v2
    glVertex2f(0.8, 0.5)      # v3
    glVertex2f(0.8, 0.2)      # v4
    glEnd()
```

Drawing Primitives: Drawing Triangle Strips (GL_TRIANGLE_STRIP) (3/4)

- **Explanation**
  - glBegin(GL_TRIANGLE_STRIP): Triangles are (v0,v1,v2), (v1,v2,v3), (v2,v3,v4)
  - With 5 vertices, we get 3 triangles forming a shape
- **Example use cases: Rendering height maps, ribbons, or any long mesh**

Drawing Primitives: Drawing Triangle Strips (GL_TRIANGLE_STRIP) (4/4)

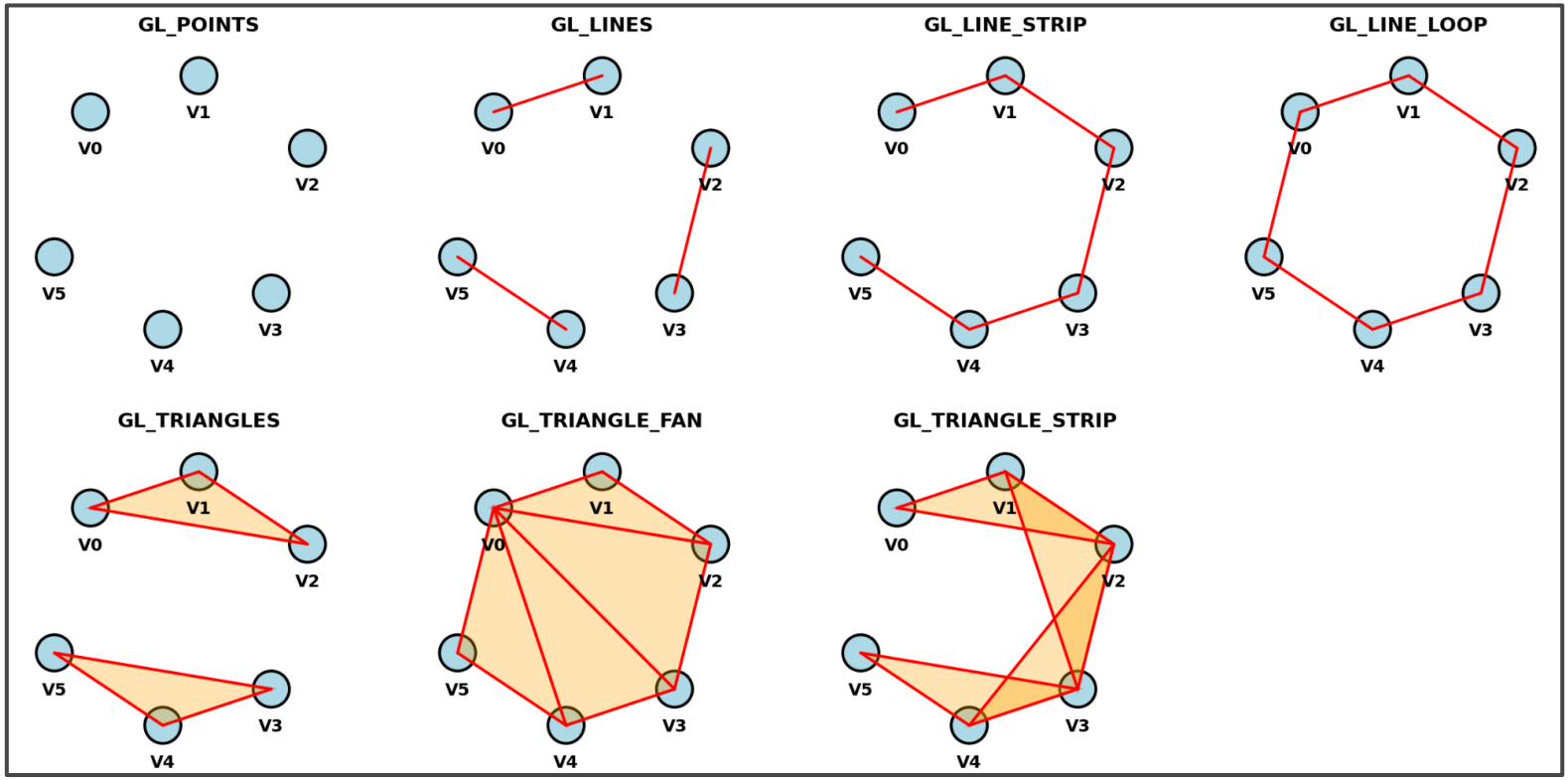


Drawing Primitives: Complete Immediate Mode Drawing Function (1/2)

- Complete fxn_draw_primitives() with all examples combined

  # See CSC 4505 github repository

Drawing Primitives: Complete Immediate Mode Drawing Function (2/2)



Lecture Series Outline

- Recap – The Graphics Pipeline
- Vertex Attributes – What Makes Up a Vertex?
- Understanding the Coordinate System on the Pygame Canvas
- Specifying 3D Vertices in OpenGL (Immediate Mode)
- Drawing Primitives – Points, Lines, and Triangles
- Building Complex Objects – A 3D Cube
- Complete 3D Example with Code Explanation
- Next Steps in the Pipeline
- Code Repository and File List

Drawing Primitives: Building a 3D Cube (1/12)

- Now we combine what we've learned to draw a cube
- A cube has 8 vertices and 6 faces. Each face is a square; we draw each face as two triangles
- We'll assign different colours to each face for clarity

Drawing Primitives: Building a 3D Cube (2/12)

- Cube vertices (centered at origin, side length 1)

  V0: (-0.5, -0.5, -0.5) (back-bottom-left)
  V1: (0.5, -0.5, -0.5) (back-bottom-right)
  V2: (0.5, 0.5, -0.5) (back-top-right)
  V3: (-0.5, 0.5, -0.5) (back-top-left)
  V4: (-0.5, -0.5, 0.5) (front-bottom-left)
  V5: (0.5, -0.5, 0.5) (front-bottom-right)
  V6: (0.5, 0.5, 0.5) (front-top-right)
  V7: (-0.5, 0.5, 0.5) (front-top-left)

Drawing Primitives: Building a 3D Cube (3/12)

- **We will draw each face with GL_TRIANGLES**
  - Vertex order matters for later back-face culling; we list them counter-clockwise when viewed from outside
  - Front face (z = +0.5) – red

```
glColor3f(1.0, 0.0, 0.0)          # Red
glBegin(GL_TRIANGLES)
# First triangle (V4, V5, V6)
glVertex3f(-0.5, -0.5, 0.5)   # V4
glVertex3f( 0.5, -0.5, 0.5)   # V5
glVertex3f( 0.5,  0.5, 0.5)   # V6
```

Drawing Primitives: Building a 3D Cube (4/12)

- **We will draw each face with GL_TRIANGLES**
  - Vertex order matters for later back-face culling; we list them counter-clockwise when viewed from outside
  - **Front face (z = +0.5) – red**

```
# Second triangle (V4, V6, V7)
glVertex3f(-0.5, -0.5, 0.5)   # V4
glVertex3f( 0.5,  0.5, 0.5)   # V6
glVertex3f(-0.5,  0.5, 0.5)   # V7
glEnd()
```

Drawing Primitives: Building a 3D Cube (5/12)

- **Explanation**
  - glColor3f(1,0,0): Sets colour to red for the entire front face
  - Two triangles cover the square: (V4,V5,V6) and (V4,V6,V7)
  - Each vertex is specified with glVertex3f. The order ensures both triangles are counter-clockwise

Drawing Primitives: Building a 3D Cube (6/12)

- **Back face (z = -0.5) – green**

```
glColor3f(0.0, 1.0, 0.0)          # Green
glBegin(GL_TRIANGLES)
# V0, V2, V1
glVertex3f(-0.5, -0.5, -0.5)  # V0
glVertex3f( 0.5,  0.5, -0.5)  # V2
glVertex3f( 0.5, -0.5, -0.5)  # V1
```

Drawing Primitives: Building a 3D Cube (6/12)

- **Back face (z = -0.5) – green**

```
# V0, V3, V2
glVertex3f(-0.5, -0.5, -0.5)  # V0
glVertex3f(-0.5,  0.5, -0.5)  # V3
glVertex3f( 0.5,  0.5, -0.5)  # V2
glEnd()
```

Drawing Primitives: Building a 3D Cube (7/12)

- **Left face (x = -0.5) – blue:**

```
glColor3f(0.0, 0.0, 1.0)          # Blue
glBegin(GL_TRIANGLES)
# V0, V3, V7
glVertex3f(-0.5, -0.5, -0.5)  # V0
glVertex3f(-0.5,  0.5, -0.5)  # V3
glVertex3f(-0.5,  0.5,  0.5)  # V7
```

Drawing Primitives: Building a 3D Cube (7/12)

- **Left face (x = -0.5) – blue:**

```
# V0, V7, V4
glVertex3f(-0.5, -0.5, -0.5)  # V0
glVertex3f(-0.5,  0.5,  0.5)  # V7
glVertex3f(-0.5, -0.5,  0.5)  # V4
glEnd()
```

Drawing Primitives: Building a 3D Cube (8/12)

- **Right face (x = +0.5) – yellow:**

```
glColor3f(1.0, 1.0, 0.0)          # Yellow
glBegin(GL_TRIANGLES)
# V1, V5, V6
glVertex3f( 0.5, -0.5, -0.5)  # V1
glVertex3f( 0.5, -0.5,  0.5)  # V5
glVertex3f( 0.5,  0.5,  0.5)  # V6
```

Drawing Primitives: Building a 3D Cube (9/12)

- **Right face (x = +0.5) – yellow:**

```
# V1, V6, V2
glVertex3f( 0.5, -0.5, -0.5)  # V1
glVertex3f( 0.5,  0.5,  0.5)  # V6
glVertex3f( 0.5,  0.5, -0.5)  # V2
glEnd()
```

Drawing Primitives: Building a 3D Cube (10/12)

- **Top face (y = +0.5) – cyan:**

```
glColor3f(0.0, 1.0, 1.0)          # Cyan
glBegin(GL_TRIANGLES)
# V3, V2, V6
glVertex3f(-0.5,  0.5, -0.5)  # V3
glVertex3f( 0.5,  0.5, -0.5)  # V2
glVertex3f( 0.5,  0.5,  0.5)  # V6
```

Drawing Primitives: Building a 3D Cube (11/12)

- **Top face (y = +0.5) – cyan:**

```
# V3, V6, V7
glVertex3f(-0.5,  0.5, -0.5)  # V3
glVertex3f( 0.5,  0.5,  0.5)  # V6
glVertex3f(-0.5,  0.5,  0.5)  # V7
glEnd()
```

Drawing Primitives: Building a 3D Cube (11/12)

- **Bottom face (y = -0.5) – magenta:**

```
glColor3f(1.0, 0.0, 1.0)          # Magenta
glBegin(GL_TRIANGLES)
# V0, V1, V5
glVertex3f(-0.5, -0.5, -0.5)  # V0
glVertex3f( 0.5, -0.5, -0.5)  # V1
glVertex3f( 0.5, -0.5,  0.5)  # V5
```

Drawing Primitives: Building a 3D Cube (12/12)

- **Bottom face (y = -0.5) – magenta:**

```
# V0, V5, V4
glVertex3f(-0.5, -0.5, -0.5)  # V0
glVertex3f( 0.5, -0.5,  0.5)  # V5
glVertex3f(-0.5, -0.5,  0.5)  # V4
glEnd()
```

Lecture Series Outline

- Recap – The Graphics Pipeline
- Vertex Attributes – What Makes Up a Vertex?
- Understanding the Coordinate System on the Pygame Canvas
- Specifying 3D Vertices in OpenGL (Immediate Mode)
- Drawing Primitives – Points, Lines, and Triangles
- Building Complex Objects – A 3D Cube
- Complete 3D Example with Code Explanation
- Next Steps in the Pipeline
- Code Repository and File List

Next Steps in the Pipeline

- **Now that we can specify vertices and draw 3D objects, the next stage is Vertex Processing**
- **Upcoming lectures**
  - Transformations (translation, rotation, scaling) – moving objects in the scene
  - Viewing – setting up a virtual camera
  - Projection – creating perspective and orthographic views
  - Clipping – removing parts of objects outside the view volume
- **Then we shall study rasterization algorithms (line drawing, circle drawing, polygon filling) – the bridge between vertex specification and the final image**

Lecture Series Outline

- Recap – The Graphics Pipeline
- Vertex Attributes – What Makes Up a Vertex?
- Understanding the Coordinate System on the Pygame Canvas
- Specifying 3D Vertices in OpenGL (Immediate Mode)
- Drawing Primitives – Points, Lines, and Triangles
- Building Complex Objects – A 3D Cube
- Complete 3D Example with Code Explanation
- Next Steps in the Pipeline
- Code Repository and File List

Code Repository and File List (1/2)

- All code examples from this lecture are available on GitHub

Module	Contents
csc4505_lecture5_template.py	Base Pygame/OpenGL window template
csc4505_lecture5_points.py	GL_POINTS example – four white points
csc4505_lecture5_lines.py	GL_LINES example – two green diagonals
csc4505_lecture5_line_strip.py	GL_LINE_STRIP example – yellow zigzag
csc4505_lecture5_line_loop.py	GL_LINE_LOOP example – cyan triangle outline
csc4505_lecture5_triangles.py	GL_TRIANGLES example – coloured triangle
csc4505_lecture5_triangle_fan.py	GL_TRIANGLE_FAN example – blue quadrilateral
csc4505_lecture5_triangle_strip.py	GL_TRIANGLE_STRIP example – magenta shape

Code Repository and File List (2/2)

- All code examples from this lecture are available on GitHub

Module	Contents
csc4505_lecture5_cube.py	3D cube with coloured faces
csc4505_lecture5_all_primitives.py	All primitives combined

Q & A Session

- Comments, concerns and complaints?

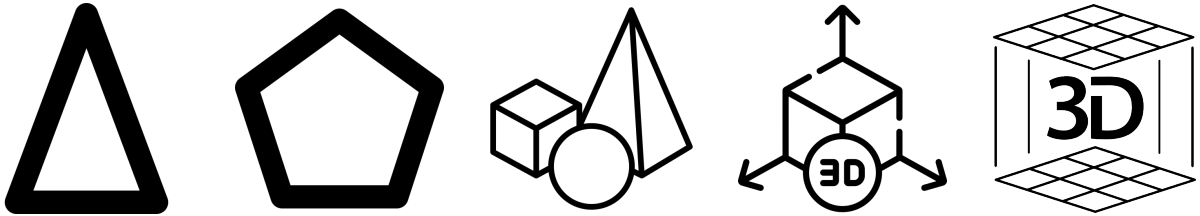
Bibliography

1. **Khronos OpenGL Wiki**
   https://www.khronos.org/opengl/wiki
2. **OpenGL Registry**
   ttps://www.khronos.org/registry/OpenGL
3. **Learn OpenGL**
   https://learnopengl.com
4. **Python & OpenGL for Scientific Visualization**
   https://www.labri.fr/perso/nrougier/python-opengl

csc4505@unza.zm

https://moodle.unza.zm/course/view.php?id=626

YouTube Playlist ID: PLubb3AIPdPhYGt3ax8orjiDo3cuK0tGfx



CSC 4505 (2025/26)
Graphics and Visual Computing

Lecture 5: Vertex Specification

Lighton Phiri <lighton.phiri@unza.zm>
Department of Computing and Informatics
University of Zambia
https://datalab.unza.zm/people/lightonphiri